Sept 6th

- In you project implement:
    - Load balancing:
    - Access Transparency: system direct user to working server, user should not feel any problem if server fails
    - Fault tolerance:

Sept 9th

- Project should be Peer to peer system, allowing users to talk directly.
- Under cloud you will do Processing, registration and discovery of service.
- We should document the progress and different alternatives, what kind of workloads you subject your system under to test whether we need objectives or not.
- Crucial to capture all details in report: experiments we have done, how you exaluated, how long you conducted workload and assess and colected data. All these need to be documented.
- How u mesure and document are listed in project description.
- Check how doctor is going to grade the project, take it into consideration.
- Use open-source systems, but you have to give credits and state where you got it from.
- Check survey we submited to see the Distributed election algorithm we chose.

Sept 13th

- When implementing Distributed election algorithm chosed, make sure leader does not play role of coordinator between nodes; doctor doesn't want to see coordinator; he wants peer to peer system.
- Make system provide remote invocation mechanism
- Request-reply, RPC or RMI ?
- On cumputers, 6 nodes, doctor wanna see one fails, and how system manage transparency and how the failed server recover, and how to handel turning off one of nodes. And how to deal with one node turned off and one failed, then only server exist; how it is going to handles 3 clients nodes.
- Handle performance loss (degradation in performance) --> load balancing
- Challenge 1: How to detect if there is a failure? Typically 5 retries then if still no response then send error
- Design cloud of 3 physical nodes, utilise dis election algo to simulate failure (node shut down) after 20 seconds should come again, and if it should be replica of another should be updated (synchronized)

- Technique: when node rejoin the system should start from where others stopped meaning should be updated.
- Implement you cloud in which alouds replicas to exist, the msg sent once at application layer, and the middleware will ensure that the replicas get the copy of the msg (make sure to distinguish between these two layers in implementation application layer is diferent from middleware layer)
- How to ensure having replicas ?
- In the slides we learn different things, but in project we need to be careful what are we choosing for our project, doctor mentions sometimes what we are supposed to use so be careful!!!

Oct 4th

- Physical layer is responsible in not making msgs get lost,
- Request reply protocol + UDP
- For project use UDP -->don't go this low level, u will use ready made RPC or RMI technique pick one and document why u picked the one u chose?

Oct 28th

- **During implementation of project collect data and report on In technical report**
- **Mimic failure using election, where u set one node to fail for 20s and then recover and foloow with other nodes from where they start (synchronization)**
- **Explain in report, how node get recover**
- **Check entries table, make sure in demo to show 4 first entries + last entry -- > 5 in total**
- **Wanna see how our system will handel this failure: physically turn off one machine**
- **He insists cloud nodes should be peers no coordinator.**
- **Documentation of all of this + our decision making " what happened", " what we did" + "who did what"**
- **Implement publsih-subscribe**
- **Implement group communication**
- **Discovery of services should be implemented in phase 2.**
- **Fe table in chapter 2, where we need to pick one system like either JAVA RMI, CORBA; Sun XDR ... check slides**